

**RESIDÊNCIA TECNOLÓGICA EM MICROELETRÔNICA - CI EXPERT - POLO
UFCG - UNIVERSIDADE FEDERAL DE CAMPINA GRANDE
RELATÓRIO DE ATIVIDADE AVALIATIVA
CONTROLADOR DE *VENDING MACHINE* EM SYSTEMVERILOG**

Aluno: Joseph dos Santos Silva, Luís Guilherme de Souza Coutinho.

Trilha: Design RTL.

Professor: Henrique do Nascimento Cunha.

1. Introdução

O presente relatório propõe-se a apresentar uma parte do desenvolvimento de um controlador digital de uma *vending machine*, apresentando o fluxo da lógica do sistema com diagramas de blocos e de estados, descrevendo de modo resumido seu código-fonte modularizado, a realização de testes com situações específicas e anotações das formas de onda, além da síntese, geração da *netlist* do projeto, e exploração dos limites do design RTL. E, por fim, finalizando com uma conclusão sobre os aprendizados e desafios da primeira parte do desenvolvimento desse controlador digital.

1.1 Repositório no GitHub

O repositório público no GitHub desta atividade, onde encontra-se o código-fonte dos módulos em SystemVerilog e dos arquivos de teste e configurações, pode ser acessado no seguinte endereço: <<https://github.com/luisign/atividade-vending-machine>>.

2. Diagramas

A seguir, mostra-se na Figura 1 o funcionamento geral do design por meio de diagramas de blocos, onde observa-se de forma geral o caminho que os dados percorrem e suas variáveis envolvidas, como entradas de usuário e saídas do sistema, a lógica combinacional de alguns blocos (exemplo: subtrator, comparador, etc.) e o controle registrado com *clock/reset* para outros blocos (exemplo: registrador de crédito, unidade de controle). Além disso, representa-se também em bloco a unidade de controle regida pelo conceito de máquina de estados finitos do tipo Moore.

Já na Figura 2, aborda-se cada estado da máquina (IDLE, COLLECT, CHECK, DISPENSE, CHANGE, ERROR) e o fluxo de decisão tomada entre os estados da máquina, como a inserção de moedas ($\text{coin_in}[1:0] \neq 0$) e assim a mudança de estado de IDLE para o estado COLLECT, ou como no estado de CHECK que procura o estoque do item e o preço para verificar se é possível realizar a compra.

Figura 1

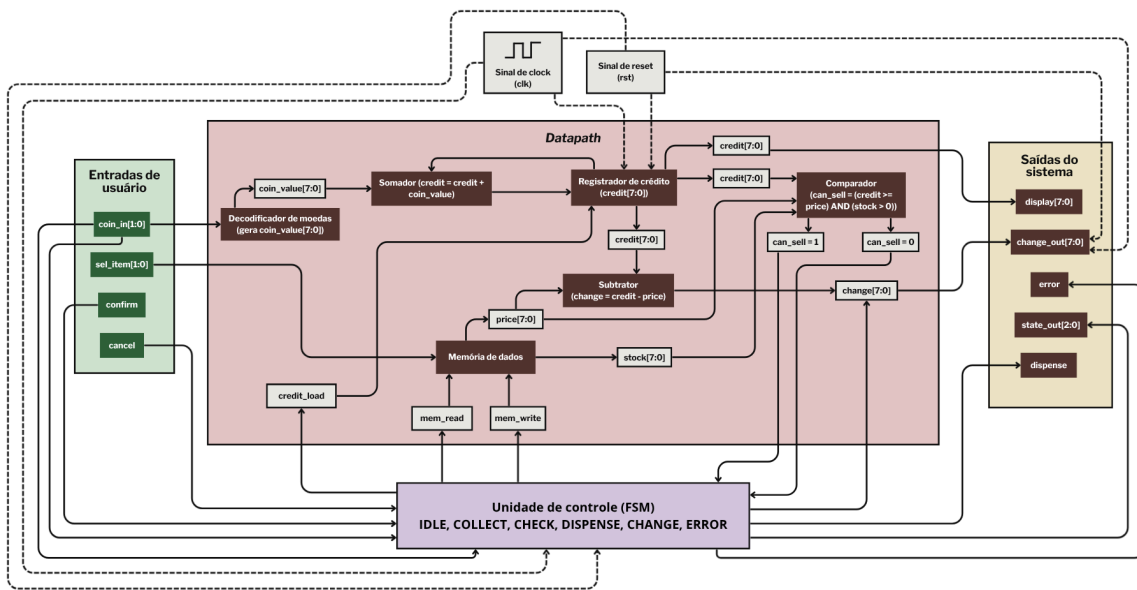


Diagrama de blocos

Figura 2

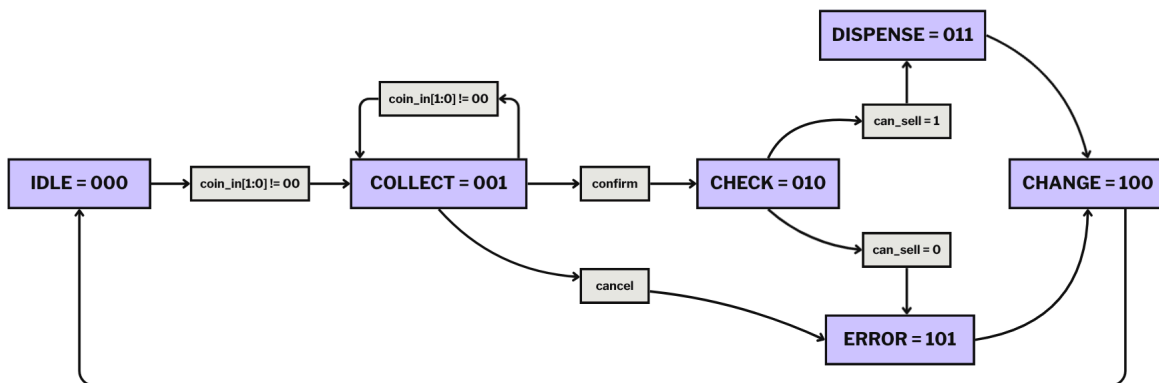


Diagrama de estados da FSM

3. Apresentação dos módulos

3.1 Módulo vending_pkg.sv

O módulo **vending_pkg.sv** cria um pacote (*package*) para o sistema, centralizando constantes, parâmetros e definições de tipos que serão compartilhados pelos módulos do design do controlador.

Dentro do pacote, o trecho que é mais importante é a criação do **state_t**, que padroniza os nomes e a codificação binária dos 6 estados da FSM. O módulo também estabelece os parâmetros **DATA_W** (onde define-se o barramento de dados em 8 bits para representar valores de crédito, preços e troco) e **SEL_W** (onde define-se 2 bits para a seleção dos 4 itens da máquina):

```
typedef enum logic [2:0] {
    IDLE      = 3'b000,
    COLLECT   = 3'b001,
    CHECK     = 3'b010,
    DISPENSE  = 3'b011,
    CHANGE    = 3'b100,
    ERROR     = 3'b101
} state_t;
```

3.2 Módulo **credit_reg.sv**

O módulo **credit_reg.sv** atua como o registrador de saldo da máquina de vendas, que armazena de modo síncrono crédito atual do usuário. Ele recebe o valor de cada moeda e por meio da unidade de controle (FSM) atualiza e mantém esse total disponível no barramento **credit[7:0]** para que o sistema possa compará-lo com os preços dos produtos e atualizá-lo no *display*.

A lógica principal do módulo utiliza um bloco **always_ff**. No funcionamento do sistema, o sinal síncrono **clear** tem prioridade para zerar o saldo e o sinal **credit_load** habilita a adição do valor da nova moeda **coin_value** ao saldo.

3.3 Módulo **memory.sv**

O módulo **memory.sv** armazena os parâmetros dos produtos e controla o inventário da *vending machine*. Ele implementa duas matrizes de registradores de 8 bits (**prices_array** e **stocks_array**) para guardar os respectivos preços e quantidades disponíveis de cada um dos 4 produtos (café, água, suco e *snack*).

A leitura dessas informações é combinacional. Quando o sinal de seleção do item (**sel_item**) é alterado, o preço e o estoque são também alterados.

No bloco sequencial **always_ff**, com o sinal de *reset* (**rst**), carrega-se a memória com os valores padrões de fábrica para cada produto. Durante o funcionamento, caso a unidade de controle ative o sinal de escrita (**mem_write**), o circuito retira uma unidade do estoque do produto selecionado.

```
// Leitura combinacional baseada na seleção
assign price = prices_array[sel_item];
assign stock = stocks_array[sel_item];

// Atualização síncrona do estoque (decremento por compra)
if (mem_write && stocks_array[sel_item] > 8'd0) begin
    stocks_array[sel_item] <= stocks_array[sel_item] - 8'd1;
end
```

3.4 Módulos **comparator.sv** e **subtractor.sv**

No módulo combinacional **comparator.sv** analisa-se se o saldo atual do usuário é igual ou superior ao preço do produto selecionado e se o estoque desse

item é maior que zero. Quando ambas as condições são verdadeiras, ele ativa a saída **can_sell** para a unidade de controle prosseguir com a compra.

Já o módulo **subtractor.sv**, também combinacional, calcula o troco após uma operação com uma subtração aritmética simples de 8 bits entre o preço do item escolhido e o crédito total acumulado pelo usuário. O resultado é a saída **change**.

3.5 Módulo control_unit.sv

No módulo **control_unit.sv** implementa-se a máquina de estados finitos do tipo Moore. O **control_unit** se divide em três blocos: um sequencial síncrono **always_ff** que é responsável por atualizar o estado atual a cada borda de subida do *clock*, e dois blocos combinacionais **always_comb**, onde um calcula a lógica de transição para o próximo estado e o outro decodifica as saídas de controle que ativam os demais módulos do *datapath*.

A máquina inicia em **IDLE** e passa para **COLLECT** assim que moedas são inseridas. A partir daí, aguarda a decisão do usuário para prosseguir para o estado **CHECK**. Com base no sinal **can_sell** (vindo do comparador), a FSM decide se o fluxo avança para a liberação do produto e cálculo do troco (**DISPENSE** e **CHANGE**) ou se desvia para a falha no estado **ERROR**.

3.6 Módulo vending_top.sv

O módulo **vending_top.sv** interconecta os módulos (submódulos) vistos até aqui no **ponto 3**. No **vending_top** mapeia-se as entradas e saídas físicas e realiza-se lógicas combinacionais principais, como a do decodificador que converte o código de 2 bits das moedas inseridas (**coin_in**) em seus valores numéricos correspondentes e a atribuição do saldo do usuário para o barramento do *display*.

Outra função do módulo é o gerenciamento síncrono da saída de troco (**change_out**). O módulo implementa um multiplexador e um registrador para isolar o valor correto a ser devolvido. Se o usuário fizer o cancelamento, a lógica faz com que o saldo acumulado (**credit**) seja devolvido, e se for uma compra válida, ele seleciona o valor calculado pelo bloco subtrator (**change_wire**).

```
// Decodificação combinacional de moedas e seleção do troco
always_comb begin
    case (coin_in)
        2'b01: coin_value = 8'd25;
        2'b10: coin_value = 8'd50;
        2'b11: coin_value = 8'd100;
        default: coin_value = 8'd0;
    endcase
end

// Definição do mux
assign mux_out = (sel_change || state_out == CHANGE && cancel) ?
    credit : change_wire;
```

4. Criação do *testbench*

O *testbench* **tb_vending.sv** criado utiliza tarefas automatizadas (*tasks*) para as rotinas de inserção de moedas e seleção de produtos. A validação das saídas do hardware ocorre por meio da tarefa **check**, que compara os sinais com os resultados esperados e exibe mensagens de aprovação (**[PASS]**) ou falha (**[FAIL]**) no terminal da simulação:

```
task check(input [7:0] expected, input [7:0] actual, input string
label);
    if (expected === actual) $display("[PASS] %s", label);
    else $display("[FAIL] %s | Esperado: %0d, obtido: %0d",
label, expected, actual);
endtask
```

O primeiro cenário simula uma compra bem-sucedida com necessidade de troco. O teste seleciona o produto da posição **00** (café) e injeta um valor elevado para cobrir o custo (R\$ 1,00). O *testbench* monitora as transições da FSM e valida se o **dispense** é ativado, e confirma se o circuito calcula o valor de R\$ 0,75 de troco, e zerar o *display* de saldo.

O segundo cenário analisa o comportamento de proteção do circuito quando ocorre uma tentativa de compra com crédito insuficiente. A simulação seleciona o produto **11** (*snack*), mas insere um valor insuficiente para compra (R\$ 0,25). O fluxo da FSM bloqueia a venda, direcionando a máquina para a posição de erro.

O terceiro cenário avalia o sistema quanto ao cancelamento da operação por parte do usuário durante o acúmulo de saldo. O *testbench* insere duas moedas consecutivas e ativa o sinal de interrupção em nível alto por dois ciclos de *clock*. A rotina valida se o circuito interrompe o processo, retorna para IDLE e realiza a devolução das 200 unidades de crédito (R\$ 2,00).

O quarto cenário do *testbench* valida o travamento quando o inventário de um produto é esgotado na memória. Um laço de repetição força a compra do item **00** (café) por quatro vezes seguidas para zerar o seu estoque. Na quinta tentativa de compra, confirma-se se a lógica combinacional barra a liberação e aciona o erro por falta de estoque.

É possível visualizar o arquivo completo do *testbench* na pasta **sim/** do repositório público do GitHub no ponto **1.1**.

5. Logs do *testbench* e waveforms com Verdi

A saída para cada cenário no terminal, após compilar os módulos e o *testbench* na lista do arquivo **arquivos.f** da pasta **rtl/** com a ferramenta VCS, e executar o arquivo **simv**:

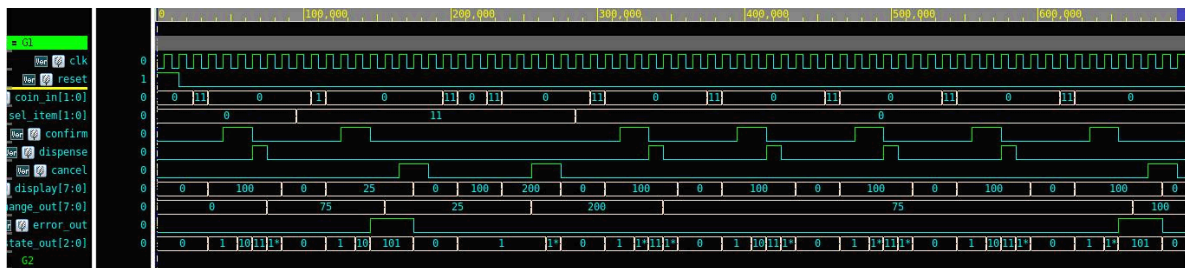
Figura 3

```
[luis.coutinho@srv-microeletronica3 rtl]$ ../sim/simv -l ../sim/sim.log
Info: [VCS_SAVE_RESTORE_INFO] ASLR (Address Space Layout Randomization) is detected on the machine. To enable $save functionality, ASLR
will be switched off and simv re-executed.
Please use '-no save' simv switch to avoid re-execution or '-suppress=ASLR_DETECTED_INFO' to suppress this message.
Chronologic VCS simulator copyright 1991-2025
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP2_Full64; Runtime version X-2025.06-SP2_Full64; Jul 10 10:48 2026
[PASS] Cenario 1: dispense = 1
[PASS] Cenario 1: troco = 75
[PASS] Cenario 1: credito = 0 ao final
[PASS] Cenario 2: ERROR = 1
[PASS] Cenario 2: FSM em ERROR
[PASS] Cenario 3: cancel troco = 200
[PASS] Cenario 3: credito = 0 apos cancelamento
[PASS] Cenario 3: FSM retorna a IDLE
[PASS] Cenario 4: ERROR estoque vazio = 1
[PASS] Cenario 4: FSM em ERROR (sem estoque)
Todos os cenarios foram validados!
$finish called from file "../sim/tb_vending.sv", line 166.
$finish at simulation time 695000
VCS Simulation Report
Time: 695000 ps
CPU Time: 0.380 seconds; Data structure size: 0.0Mb
Fri Jul 10 10:48:24 2026
```

Captura de tela do terminal Linux com os *reports* PASS em cada cenário

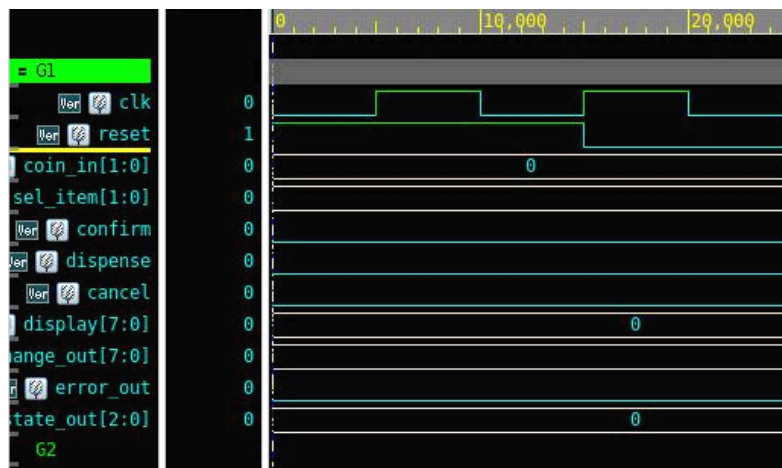
No Verdi, as formas de onda são apresentadas da seguinte forma:

Figura 4



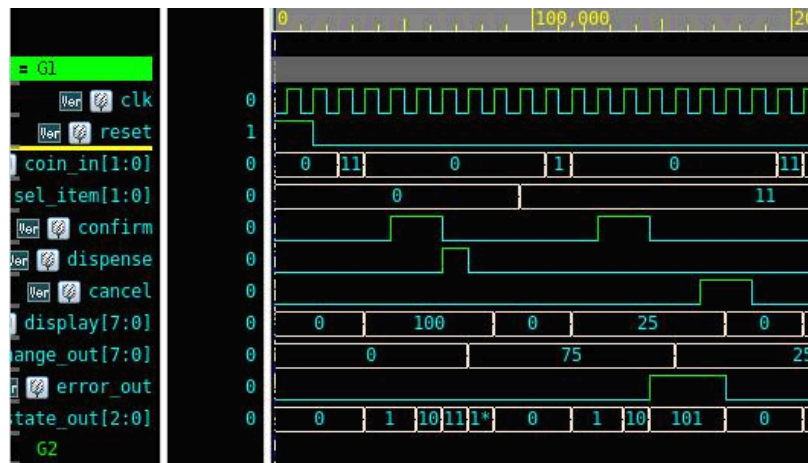
Captura de tela do testbench completo no Verdi

Figura 5



Formas de onda do início do sinal de **reset**

Figura 6



Formas de onda da compra com troco

Figura 7



Formas de onda com tentativa de compra com saldo insuficiente

Figura 8



Formas de onda com cancelamento com saldo

Figura 9



Formas de onda do estoque zerado

6. Resultados de síntese

Após rodar o *script* de síntese no DC shell, foram gerados na pasta **synth/reports/** os arquivos **check_design.rpt**, **report_area.rpt**, **report_constraint.rpt**, **report_power.rpt** e **report_timing.rpt**. A partir deles, foram analisados os resultados de *slack* (a diferença entre o momento em que o sinal precisa chegar a um registrador e o momento em que ele chega após passar pelas portas lógicas) de 15.45ns e área total do circuito de aproximadamente 862.57 micrômetros quadrados. Com o *slack* positivo (MET), sabe-se que o sinal chegou antes do prazo.

7. Exploração dos limites do design

A seguir, a tabela detalhando o comportamento do circuito sob diferentes restrições de período:

Período	Frequência (MHz)	Slack(ns)	Área	Timing
20.0	50.0	15.45	862.578539	MET
18.0	55.5	13.45	862.578539	MET
16.0	62.5	11.45	862.578539	MET
14.0	71.4	9.45	862.578539	MET
12.0	83.3	7.45	862.578539	MET
10.0	100.0	5.45	862.578539	MET
8	125.0	3.45	862.578539	MET
6	166.6	1.45	862.578539	MET
4	250.0	-0.09	1174.12856	VIOLATED

Após retirar o **-no-autoungroup** do script de síntese, com o período de 20ns e frequência de 50MHz, o *slack* ficou 15.49ns e a área total ficou 769.570343. Em suma, houve redução da área total.

Com o *timing* violado no período de 4ns e frequência de 250MHz, o script de síntese foi rodado novamente e o *slack* continuou negativo, com -0.06. Portanto, o período deve ficar 6ns ou acima.

8. Análise e discussão

8.1 Das restrições de período

De 20ns a 6ns de período, a área total permaneceu 862.57 micrômetros quadrados. Ela só variou quando o *slack* ficou negativo e o *timing* foi violado. Teoricamente, a área deve aumentar progressivamente ao diminuir o período. Na restrição de período de 4ns, a área total subiu para 1174.12 micrômetros quadrados. O Design Compiler realiza o redimensionamento do circuito para trocar as portas lógicas normais por células de alta potência pela frequência mais alta que o circuito deve atender na situação.

Percebendo-se pela tabela, o Design Compiler falhou em fechar o *timing* do projeto na frequência de 250 MHz (período de 4,0 ns). Nesse ponto de quebra, o *slack* torna-se negativo, registrando -0,09 ns. Esse resultado indica uma violação física de tempo de *setup*.

8.2 Caminho crítico

Uma análise importante a ser feita ao fim da síntese RTL é a decisão do caminho crítico. Foi percebido que o caminho crítico do design está localizado na lógica de transição da venda. Inicia-se na lógica sequencial inicial nos registradores da unidade de controle (máquina de estados), passa pelos blocos de memória interna para consulta e atualização dos itens, atravessa os comparadores e subtratores de 8 bits do *datapath* e finaliza na lógica combinacional de próximo estado da FSM. Ele é crítico porque o sinal precisa atravessar todos os blocos citados antes que o próximo pulso de *clock* aconteça.

Portanto, a transição de estado da máquina de estados que está no caminho crítico no design é o **ST_CHECK/CHECK**, quando a transição decide se o fluxo irá para a liberação do produto **ST_DISPENSE** ou para a tela de erro **ST_ERROR**. O motivo é o acúmulo de operações sequenciais em um único ciclo de *clock*, já que o circuito precisa ler a memória, comparar o crédito com o preço, checar se há estoque e calcular o próximo estado de uma só vez.

Para reduzir o caminho crítico, a modificação no RTL seria a inserção de registradores de *pipeline* nas saídas dos comparadores. Em vez de fazer o cálculo e a transição da FSM na mesma etapa combinacional, o *datapath* realiza as comparações de saldo e estoque um ciclo antes, e salva os resultados em *flags* de 1 bit registradas. No estado **ST_CHECK**, a FSM avaliará apenas duas variáveis

booleanas simples, e isso permite que o chip possa operar em frequências mais elevadas.

9. Conclusão

Conclui-se que o projeto de um controle digital de vending machine chegou ao resultado esperado para as especificações. O processo exigiu inicialmente a correção da sintaxe no Vlogan, a resolução de problemas de compilação no VCS e problemas com a visualização das formas de ondas no Verdi. Houve dificuldade para ajustar o *testbench* para os resultados esperados, além de muitas outras dificuldades utilizando os comandos no terminal Linux e das ferramentas. Em suma, as dificuldades foram superadas e contribuíram de forma significativa para o aprendizado dos discentes na residência tecnológica do CI Expert.